
Introduction to Machine Learning

Contact Session 1 — Module 1

What does it really mean for a machine to “learn”? We build the whole idea from scratch, in plain words, with everyday examples.

- | | |
|------------------|------------------|
| ■ Intuition | ■ Analogy |
| ■ Worked example | ■ Solved problem |
| ■ Common pitfall | ■ Key takeaways |

Contents

1	The one big idea	3
2	So what <i>is</i> machine learning?	4
2.1	When is learning the right tool — and when not?	4
3	The words you’ll hear over and over	6
4	The three families of learning	7
5	How to set up a learning system (a 4-question checklist)	8
6	The leap of faith behind it all	9
7	What makes it hard (the honest list)	10
8	Putting it all together (a tiny example)	11
9	The whole story on one page	12

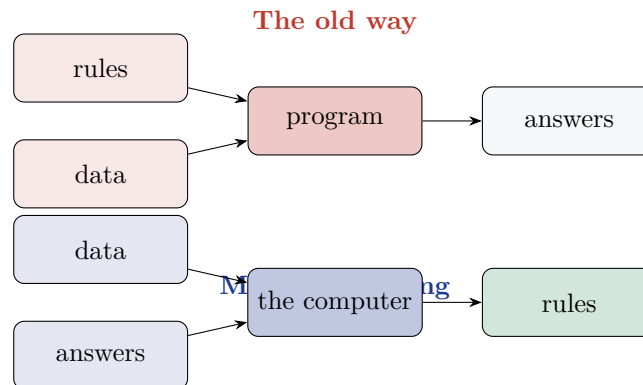
1 The one big idea

Here is a puzzle. You want a program that spots spam e-mail. The old way is to sit down and write rules by hand: “if it says *free*, flag it; if it says *prize*, flag it.” But spammers keep changing their words, so your list of rules never ends and never quite works.

Machine learning flips this around. Instead of you writing the rules, you *show the computer lots of examples* — thousands of e-mails already marked “spam” or “not spam” — and let it **figure out the rules by itself**.

★ Learning is programming, backwards

Normally you give a computer *rules* and *data*, and it gives you *answers*. In machine learning you give it *data* and *answers*, and it gives you back the *rules*. You stop writing the logic and start collecting good examples instead. That single switch is the whole idea.



2 So what *is* machine learning?

A famous one-liner (from Tom Mitchell) sounds heavy but is actually simple once you unpack it:

A program learns if it gets better at a task, measured by some score, as it gets more experience.

Three plain words hide inside: the **task** (what we want done), the **score** (how we check if it's doing well), and the **experience** (the examples it learns from). Pin down those three and you have a real learning problem.

✓ Three everyday examples, three plain pieces

What we want	The task	The score	The experience
Catch spam	label mail spam / not	% labelled correctly	old mail, already labelled
Read handwriting	turn a scribble into letters	% of words read right	photos of handwriting
Win at checkers	play a good game	% of games won	games it plays against itself

! Common Pitfall

“Make e-mail better” is *not* a learning problem — it's too vague. Until you can say plainly what the task is, how you'll *score* it, and what *examples* it learns from, there is nothing to train. Always nail down those three first.

2.1 When is learning the right tool — and when not?

Writing rules by hand is fine for, say, adding up a bill. Learning earns its keep when:

- **Nobody knows the rules.** How exactly do you recognise a friend's face? You just *can*, but you can't write the steps down. Let the machine learn from examples.
- **The rules keep changing.** Spam, fashion, fraud — the pattern shifts, so a fixed rule-list rots. A learner can simply re-learn from fresh examples.
- **Every case is different.** Personalised recommendations, personalised medicine — one rule can't fit everyone.

Use machine learning when **(1)** there really is a pattern, **(2)** you can't write that pattern down as a neat rule, and **(3)** you have examples to learn from. If any one is missing, don't reach for it.

★ Where do AI, ML and “deep learning” fit?

Picture three boxes nested inside each other. The big outer box is **Artificial Intelligence** — the dream of machines that act smart. Inside it sits **Machine Learning** — one way to get there, by learning from data. Inside *that* sits **Deep Learning** — machine learning done with big brain-like networks. So deep learning is a kind of machine learning, which is a kind of AI.

3 The words you'll hear over and over

Most data is just a table. Knowing two words lets you read any of them.

features the facts we know			label the answer
CGPA	Skills	Aptitude	Got a job?
9.1	Average	Good	Yes
8.4	Good	Good	Yes
8.3	Poor	Average	No

Each **row** is one *example* (here, one student). Each **column** is one *feature* — a fact we know. In learning-from-answers problems, one special column is the **label** — the answer we want to predict. That's the whole vocabulary you need to start.

4 The three families of learning

Machine learning splits into three big families, and the difference is simply *what kind of help the computer gets*.

Learn from answers
(supervised)

Find structure
(unsupervised)

Learn from rewards
(reinforcement)

Learn from answers (supervised). We give the computer examples *with* the correct answers and ask it to predict answers for new cases. Two kinds: if the answer is a *number* (a price), it's **regression**; if the answer is a *category* (spam / not), it's **classification**.

Find structure (unsupervised). We give examples but *no* answers, and ask the computer to find natural groups or patterns on its own — for example, sorting customers into “big spenders” and “small spenders” without anyone labelling them.

Learn from rewards (reinforcement). The computer tries things, gets a reward or a penalty, and slowly learns what works — the way you'd train a dog, or teach a robot to walk.

↪ Three ways to learn to cook

With answers: you cook from a recipe book where every dish is named and pictured.

Finding structure: someone hands you a pile of ingredients and you group the dishes that seem to go together — no names given. **With rewards:** no recipe at all, but a friend tastes each attempt and says “better” or “worse,” and you improve over many tries.

★ “Supervised” just means “with answers”

Don't let the textbook words trip you. *Supervised* only means “we had the answers while teaching it.” *Unsupervised* means “no answers, find your own patterns.” That's it.

Σ Name the family

(a) Predict tomorrow's temperature. (b) Split shoppers into look-alike groups. (c) Decide if a card swipe is fraud, using past labelled swipes. (d) Teach a robot to walk by trial and error.

Answers: (a) learn-from-answers, number $\Rightarrow$ *regression*. (b) no answers $\Rightarrow$ *find structure*. (c) learn-from-answers, category $\Rightarrow$ *classification*. (d) trial and reward $\Rightarrow$ *reinforcement*. Naming the family first instantly tells you which tools even apply — always your first move.

5 How to set up a learning system (a 4-question checklist)

Suppose you're asked to build something that learns. Mitchell suggests answering four plain questions, in order. Let's use "a program that learns to play checkers" as the running example.

1. **What examples will it learn from?** Whole games? Just good moves? Will someone label the best move each turn, or will it only find out at the end who won? (Learning only from the final win/loss is harder — the computer has to guess which earlier moves deserve the credit.)
2. **What exactly should it learn to judge?** For checkers, a good target is: "look at a board and score how good it is for me." If it can score boards well, it can pick good moves.
3. **How will it write that judgement down?** The simplest choice: score a board by adding up a few facts — my pieces, my kings, my threatened pieces — each with its own weight. (Notice this is just the "multiply-and-add" line idea from regression, wearing a different hat.)
4. **How will it improve?** Play, see how its scores did, and nudge the weights to do better next time. (That nudging is exactly the "roll downhill" idea you'll meet in Module 3.)

★ Why we don't learn the "true" answer directly

We almost never get to learn the perfect answer straight away. Instead we pick a *stand-in* that's easy to learn (here, a board-scoring rule) and trust that getting good at the stand-in makes the whole system play well. Choosing a good stand-in is half the art of machine learning.

6 The leap of faith behind it all

Every “learn from answers” method rests on one quiet assumption:

If a rule works well on *enough* past examples, it will probably work well on *new* ones too — **as long as the new cases look like the old ones.**

That’s why we care so much about having *lots* of examples, and examples that *look like* the real world. If you train a spam filter only on English mail and then feed it French mail, the leap of faith breaks.

~> The student who memorised vs. the one who understood

A student who *memorises* last year’s paper aces a re-run but fails a fresh exam. A student who only learned “study hard” is too vague to answer anything. The one who learned the *ideas* sits in the sweet spot — general enough to handle new questions, sharp enough to be useful. We want our models in that same sweet spot, and the rest of the course is largely about getting them there.

7 What makes it hard (the honest list)

- **Too few examples.** Most methods are hungry; with little data they guess poorly.
- **Examples that don't match reality.** A biased sample teaches biased rules.
- **Messy data.** Typos, missing entries, weird outliers — clean them first (Module 2).
- **Useless facts.** Feed in junk features and you get junk out. Good features matter more than fancy methods.
- **Memorising vs. understanding.** Too simple a model misses the pattern; too complex a model memorises the noise. We're always hunting the middle.

8 Putting it all together (a tiny example)

✓ Guessing a used-car price, start to finish

1. **Which family?** The answer is a price (a number) $\Rightarrow$ learn-from-answers, *regression*.
2. **Examples:** past sales — each car's brand, age, mileage, and the price it fetched.
3. **What to learn:** a rule that turns those facts into a price.
4. **How to write it:** start simple — multiply each fact by a weight and add them up (Module 3).
5. **How to score it:** how far the guessed prices are from the real ones.
6. **Check honestly:** test on cars it never saw during training.

Every later module just makes one of these steps richer. The skeleton never changes.

9 The whole story on one page

★ Key Takeaways

- Machine learning **learns the rules from examples**, instead of you writing them.
- A real learning problem needs three plain pieces: a **task**, a **score**, and **experience** (examples).
- Reach for it when there's a pattern, you can't write it as a rule, and you have data.
- Three families: **learn from answers** (supervised — splits into regression for numbers, classification for categories), **find structure** (unsupervised), **learn from rewards** (reinforcement).
- In a data table: rows are **examples**, columns are **features**, the answer column is the **label**.
- To set one up, answer four questions: what examples, what to learn, how to write it, how to improve.
- It all rests on one bet: a rule that fits enough past examples will fit new ones too — if they look alike.
- The usual troubles: too little or mismatched or messy data, useless features, and memorising instead of understanding.

Where this comes from: Mitchell, *Machine Learning*, Ch. 1; Alpaydin, *Introduction to Machine Learning*, Ch. 1. Course slides M1 (Dr. Neha Vinayak, BITS Pilani WILP).